

Server:

=====

=> Server is a software which is used to run our Application.

Java Features

=====

- 1.Simple
- 2.Platform Independent
- 3.Robust
- 4.Java supports OOPs
- 5.Secure
- 6.Portable
- 7.Distributed
- 8.Dynamic language

=> What is the difference between JDK, JRE & JVM ?

JDK(Java Development Kit) = JRE + DevTools

JRE(Java Runtime Environment) = JVM +Libraries

JVM(Java Vertual Machine)

-JDK contains set of tools to develop java programs.

-JRE providing a platform to run our java programs.

-JVM will take care of program execution.

Variables : Variables are used to store the data.

Data Types: Data types are used to specify the type of data.

Identifier : The name which we are using for packages,variables,classes & methods is called as identifier.

Operator : Operator is a symbol which perform some operation on operands

EX:

int a = 3;

int b = 5;

```
int c = a + b;
```

=> What is the difference between while loop and do-while loop ?

While ==> It will check the condition first then it will execute the statements.

do-while ==> It will execute the statement first then it will check the condition.

Array: Array is a container which is used to store collection of elements with same data type.

=====

String: String is immutable(we cant change) pre-difened class in java which is used to store group of characters.

=====

String object will be created in 2 ways:

1.Literal way: It will be stored in SCP(String Constant Pool).It will not allow duplicate objects.

Ex: String str = "hello Sudha";

2.Using new Operator:It will be stored in Heap area.It will allow duplicate objects.

Ex: String str = new String("Hello sudha");

Note 1: In Strings equals() method will compare the content of the object.

Note 2: In Strings == will compare the address of the objects not the content.

Ex: String s1 = "Hello sudha";
String s2 = "Hello sudha";

here s1 and s2 contains different address but same content.

StringBuffer: StringBuffer is mutable(we can change) pre-difened class in java which is used to store group of characters. StringBuffer is thread Safe(only one person can access at a time).

StringBuilder: StringBuilder is mutable(we can change) pre-difened class in java which is used to store group of characters. StringBuilder is not thread Safe(Multiple persons can access at a time).

What is the difference between String, StringBuffer,StringBuilder ?

String : Immutable + introduced in JDK 1.0

StringBuffer : Mutable + Thread-Safe (It means two threads can't call the methods of StringBuffer simultaneously.) + introduced in JDK 1.0

StringBuilder : Mutable + Not Thread-Safe (It means two threads can call the methods of StringBuilder simultaneously.) + introduced in JDK 1.5

Command Line Arguments : Command Line Arguments are used to supply dynamic values as input for our program at the run time.

to compiling :- [javac fileName.java]

to run :- [java fileName argValue1 argValue2..] (for providing input at runtime)

OOPS :

=====

1. Encapsulation : Encapsulation means wrapping up the data and code together as a single unit. (It is a mechanism of combine the variables and methods as a

=====

single unit).

=> By using classes we can achieve encapsulation.

2. Abstraction : Abstraction means hiding internal details and showing essential functionality to the user.

=====

=> By using abstract classes and interfaces we can achieve Abstraction.

3. Polymorphism : Polymorphism means one task performed in different ways.

=====

=> By using methods we can achieve polymorphism.

=> Polymorphism is divided into 2 types:

1. Compiletime Polymorphism / Static Polymorphism :

=====

Ex: Method Overloading (The process of writing more than one method with same name and different parameters is called as Method Overloading)

===

2. Runtime Polymorphism / Dynamic Polymorphism :

=====

Ex: Method Overriding (The process of writing same method in parent class and child class is called as Method Overriding).

===

4. Inheritance : Inheritance means one class acquires all the properties and behaviour of another class.(Extending the properties from one class to another class).

this :

=====

=>: this is a pre defined keyword in java.

=>: It is used to represents current class object.

-> this keyword is to eliminate the confusion between class attributes and method or constructor parameters with the same name

-> if u have same name for class attributes(variable) and constructor or any method parameters, then compiler will get confuse to which one needs to consider, so will use [this.variable_name] inside our constructor method

super :

=====

=>: super keyword is used to access parent class

properties(variable/method/constructor). (or) To invoke method from parent class

-> if u have same method names in both parent and child classes, then compiler will get confuse like which one we should get, so by using [Super.method_name] we can get parent class method.

Constructor: It is a special method in java which is used to initialize the current class instance variables.

=====

=> : Constructor name should be same as class name.

=> : Constructor will not have any return type(not even void).

Constructor Overloading: Writing more than one constructor with different parameters is called as Constructor Overloading.

=====

Types of Relations in Java :

=====

*. In Java classes we have 2 types of relations.

1.IS-A relation: If one class wants to re-use all the properties of another class then we will go for IS-A Relation.

=====

Ex: Inheritance

2.HAS-A relation : If one class wants to re-use some properties of

another class then we will go for HAS-A Relation.

=====

Ex: Composition (Creating one class Object into another class)

Types of methods in Java :

=====

=> In Java we have 2 types of methods.

1. Concrete Method : The method which contains body is called Concrete Method.

=====

```
Ex: Public void m1() {  
    // logic  
}
```

2. Abstract Method : The method which doesn't contains body is called Abstract Method.

=====

```
Ex: Public abstract void m2();
```

Marker Interface :

=====

=> Any interface doesn't contain any method then that interface is called Marker Interface.

Ex: Cloneable,Serializable etc.

Abstract Class :

=====

=> The class which contains both concrete and abstract methods is called as Abstract class.

toString() :

=====

=> It is used to represent Object in string format.

Exception :

=====

=>: Un-expected and Un-wanted situation in the program execution is called as Exception.

=>: Exception will disturb the normal flow of the program execution.

=>: When Exception occurred program will be terminated abnormally.

Exception Handling :

=====

=>: The process of handling Exceptions is called as Exception Handling.

Throwable:

=====

=>: Throwable is the root class for Exception and Error.

Q) What is the difference between Exception and Error ?

=> Exceptions can be handled where as Errors can't be handled.

Exception Types :

=====

=> Exceptions are divided into 2 types.

1.Checked Exceptions: Will be identified at compile time.

=====

Ex: IOException, FileNotFoundException, SQLException etc.

===

2.Un-Checked Exceptions: Will be identified at run time.

=====

Ex: NullPointerException, ArithmeticException etc.

===

try block:

=====

=> It is a key word in java which is used to keep risky code.

syntax:

=====

```
try{  
    // statements  
}
```

Note: we can't write only try block.

=====

```
try-catch-----> valid combination
try-finally-----> valid combination
try-catch-finally-----> valid combination
catch-finally-----> invalid combination
only try-----> Invalid combination
only catch-----> Invalid combination
only finally-----> Invalid combination
```

catch block:

=====

=> It is a key word in java which is used to catch the exception which occurred in try block.

=> To write the catch block, try block is mandatory.

syntax:

=====

```
try{
    // statements
}catch(Exception e){
    // logic to catch exception info
}
```

Note: If Exception occurred in try block then only catch block will execute otherwise catch block will not execute.

=====

throws : It is a keyword in java which is used to ignore checked exceptions.

=====

throw : It is a keyword in java which is used to create the exception wantedly.

=====

Q) What is the difference between final, finalize(), finally ?

final: final is a keyword which is used to declare final variables, final methods, final classes.

=====

finalize(): It is a Pre-defined method in Object class, and it will be called by Garbage collector before removing Un-used objects from the Heap area.

=====

finally: It is a block in Exception handling which is executed always whether

exception occurred or not.

=====

Variable Arguments (var-args) :

=====

=> It is used when we don't know how many parameters to take for a method then we can use var-args method.

=> It is an advanced concept for method overloading.

=> var-args concept introduced in 1.5v.

syntax:

=====

```
public int add(int... x) {  
  
}
```

Note:

=====

=> Only 3 Ellipses(dots) allowed to declare variable argument.

=> A method should contain only one variable argument.

=> variable argument should be the last argument of the method.

```
Ex:      public int add(String s, int... x) {  
  
}
```

Q) What is Wrapper class in Java ?

=> The Wrapper class in Java is used to convert primitive data types into objects.

=> Wrapper classes are used to implement Auto-Boxing and Un-Boxing.

Q) What is Auto-Boxing and Un-Boxing in Java ?

=> Auto-Boxing : The process of converting primitive data type into corresponding Wrapper class is called as Auto-Boxing.

=> Un-Boxing : The process of converting Wrapper class into corresponding primitive data type is called as Un-Boxing.

```
Ex:   byte b = 20;           // primitive data type  
      System.out.println(b);
```

```
      Byte b1 = new Byte(b); // Auto-Boxing  
      System.out.println(b1);
```

```
      byte b2 = b1;          // Un-Boxing
```

```
System.out.println(b2);
```

Type-Casting : The process of converting from one data type to another data type is called as Type-Casting.

=====

=> Type-Casting is divided into 2 types.

1.Up-Casting (Widening) : The process of converting data from lower data type to higher data type is called as Up-Casting/Widening.

=> As we are converting from lower data type to higher data type there is no data loss.

=> For Widening no need to specify type-casting explicitly.

```
Ex:   byte b =20;
      int i = b;
```

2.Down-Casting (Narrowing) : The process of converting data from higher data type to lower data type is called as Down-Casting/Narrowing.

=> As we are converting from higher data type to lower data type there is a chance of data loss.

=> For Narrowing we need to specify type-casting explicitly.

```
Ex:   long num = 100;
      int x = num; // compile time error
      int x = (int)num;
```

Q) What is the difference between FileReader and BufferedReader ?

=> : FileReader will read the data character by character where as BufferedReader will read the data line by line.

IO Streams:

=====

=> : IO Streams are used to perform operations on the file.

=> : Using IO Streams we can establish link between java program and File.

```
                write
java program <-----> File
```

```
                read
java program <-----> File
```

=> IO Streams are divided into 2types:

1.Byte Stream :To read/write binary data(images, videos, audios, pdf's etc).

=====

=> Byte Stream providing 2types of classes:

- 1.Input Stream Related Classes (Ex:FileInputStream)
- 2.Output Stream Related Classes(Ex:FileOutputStream)

2.Character Stream :To read/write character data.

=====

=> Character Stream providing 2types of classes:

- 1.Reader Classes (Ex: FileReader)
- 2.Writer Classes (Ex: FileWriter)

Serialization : The Process of converting java object into file is called as Serialization.

=====

De-Serialization : The Process of converting file data back to java object is called as De-Serialization.

=====

Note: To perform Serialization or De-serialization the java class should implement java.io.Serializable marker interface.

=====

Thread:Thread is a one pre defined class available in java.lang package.

=====

=> We have 3types of threads in java.

1.Default thread created by JVM (main thread)

2.User Defined Threads(Thread class, Runnable interface, Callable interface)

3.Daemon Threads: The Thread which runs in the background is called as Daemon Thread.

=>: We created Daemon thread by using setDaemon() method.

Ex: Garbage Collector is a daemon thread

Single Tasking : Performing only one task at a time is called Single Tasking.

=====

Multi Tasking : Performing multiple task at a time is called Multi Tasking.

=====

=> Multi tasking we can achieve in 2 ways.

- 1.Process Based Multi Tasking.
- 2.Thread Based Multi Tasking.

=====

User Defined Threads:

=====

=> In java we can create Thread in 2 ways.

- 1.By extending Thread class
- 2.By implementing Runnable interface

=====

Thread Scheduler: Thread Scheduler is a program in the JVM which is responsible to schedule Threads execution and resources allocation which are required for

=====

the thread execution.

=====

Thread Life Cycle:

=====

=> Thread life cycle contains several phases of Thread execution.

- 1.New: A thread begins its life cycle in the new state. Thread remains in the new state until we call start() method.
- 2.Runnable: After calling start() method, thread comes from new state to runnable state.
- 3.Running: A thread comes to running state when thread scheduler will pick up that thread for execution.
- 4.Blocked: A thread is in waiting state if it waits for another thread to complete its task.
- 5.Terminated: A thread enters into terminated state once it complete its task.

Q)What is the difference between Runnable interface and Callable interface ?

=>: Runnable is a functional interface which contains run() method where as Callable is a functional interface which contains call() method.

=>: Runnable run() method return type is void where as callable call() method return type is Object.

=>: Runnable interface is present in java.lang package where as Callable interface present in java.util.concurrent package.

=====
ExecutorService:
=====

=>: ExecutorService introduced in java 1.5v
=>: Using ExecutorService we can implement mutli threading.
=>: Using ExecutorService we can create thread pool.
=>: Using ExecutorService we can submit the task to pool of threads.

```
Ex:      Demo d = new Demo();
         ExecutorService exServcie = Executors.newFixedThreadPool(5);
         exService.submit(d);
```

=====
Synchronization: Synchronization means allowing one only thread to execute our logic/resource/object at a time.
=====

String -----> Immutable

StringBuffer-----> mutable class + synchronized class (Thread safe)

StringBuilder-----> mutable class + non-Synchronized class (not thread safe)

=>: Synchronized means thread safe -----> only one thread can access the object at a time
=>: Non-synchronzed means not thread safe---> multiple threads can access the object at a time

=>: If multiple threads access the same object at a time, there is a chance of getting in-consistency problems.
=>: To avoid those in-consistency problems, we need to use Synchronization concept.
=>: Synchronization means allowing one only thread to execute our logic/resource/object at a time.
=>: By using synchronization we can achieve thread safety but it will slow down our execution process.

=====
How to Achieve Synchronization:
=====

=>: By using "synchronized" keyword we can achieve Synchronization.
=>: Synchronize keyword we can use at 2 places in programming.

- 1.At Method level
- 2.At Block level

Syntax for block level:

=====

```
public void m1() {  
    // pre logic  
    synchronized(object) {  
        // important business logic  
    }  
  
    // post logic  
}
```

Syntax for Method level:

=====

```
public synchronized void m1() {  
    // important business logic  
}
```

=====

Dead lock: If 2 threads are waiting for each other to release resources is called as Dead Lock.

=====

=>: Dead lock means ambiguity problem among the threads.

=>: Once we get into dead lock situation we can't do anything.

Ex:

===

Thread-1 holding resource-1 and waiting for resource-2.

Thread-2 holding resource-2 and waiting for resource-1.

Note:

=====

=>: Thread-1 will not release resource-1 hence thread-2 will be in waiting state

forever for resource-1.

=>: Thread-2 will not release resource-2 hence thread-1 will be in waiting state forever for resource-2.

join() Method : join() method is used to hold second thread execution until first thread execution got completed.

=====

yield() Method : yield() method is used to give chance to other threads to execute equally.

=====

inter communication threads:

=====

=> It is used to establish communication among the threads.

=>: To achieve inter thread communication we have 3 below methods in object class

1.wait()

2.notify()

3.notifyAll()

Reflection API:

=====

=>: By Using Reflection API we can analyze our java classes.

=>: Reflection API is used to analyze the software.

Note:

=====

=>: Private variables can access outside the class by using reflection API.

=>: In OOP's private variables can't access outside the class.

NoClassFoundError: =>: NoClassDefFoundError means class available at compile time but not available at run time.

=====

Q) What is the difference between ClassNotFoundException and NoClassDefFoundError ?

=>: If we give wrong class name in Class.forName(" ") then we will get ClassNotFoundException.

=>: If JVM will not able to find .class file of particular class then we will get NoClassDefFoundError.

Garbage Collection:

=====

=>: Garbage Collection is the process of removing un-used/un-referenced objects from the JVM's Heap area.

=>: Garbage Collection is an in-built process of JVM.

=>: In JVM, Garbage Collector is available to perform Garbage Collection.

=>: Garbage Collector is a daemon thread(runs in background).

EX:

===

```
package in.ashokit;
public class Student {

    public static void main(String[] args) {

        // Object created
        Student s1 = new Student();

        // nullifying ( making obj eligible for GC)
        s1 = null;

        System.gc();

    }

    protected void finalize() throws Throwable {
        System.out.println("Finalize method called...");
    }
}
```

=>: When we assign s1= null then s1 become un-referenced object and it is eligible for GC.

=>: When GC identified un-referenced objects then it will call finalize() method and it will remove that object from HEAP Area(memory clean up).

How to invoke GC in java:

=====

=> In java we have 2ways to invoke GC.

```
1.System.gc();
2.Runtime.getRuntime.gc();
```

Note: Even if we call above methods there is no guarantee that JVM will start GC immediately. GC execution process will be managed by JVM only.

=====

How Garbage Collection works internally in JVM :

=====

=>: Garbage Collection works in below phases

1.Stop the World: When GC starts it will trigger STOP THE WORLD (All running threads will be stopped for few milli seconds).

2.Marking: GC will go to JVM Heap Area and it will identify un-referenced objects and it will mark them.

3.Sweeping: GC will sweep marked objects.

4.Compaction: After sweeping completed memory holes will be created in Heap Area to clear that GC will perform Compaction(adjusting memory holes).

After compaction, GC will give signal to JVM to continue the execution.

Generics in Java:

=====

=>: Generics are introduced in java 1.5v.

=>: Generics are used to achieve type safety.

=>: Using generics, we can write classes/variables/methods which are independent of data type.

EX:

===

```
public class demo {  
  
    public void m1(Integer i) {  
  
    }  
  
    public static void main(String[] args) {  
        demo<Integer> d = new demo<>();  
        d.m1(10);    // valid  
        d.m1("sudha") // invalid  
    }  
}
```

Generics with Wild Cards :

=====

Syntax : Demo< ? extends Number>

=====

=>: The above wild card represents Demo class should take any class obj which is extending from the Number class.

Enums in Java :

=====

=>: Enum introduced in java 1.5v.

=>: Enum is a special data type in java.

=>: Enum data type is used to create pre-defined constants.

=>: To declare constants using Enum we will use "enum" keyword.

=>: Enum stands for Enumeration.

=>: When we want to declare some pre-defined constants then we will use Enums concept.

Ex:

```
enum Weekdays{
    monday,tuesday,wednesday,thursday,friday;
}

enum WeekendDays{
    saturday,sunday;
}
```

Class Loaders:

=====

=>: Class Loaders are used to load .class files into JVMs memory.

=>: Class Loaders are divided into 3 types in java.

1.Bootstrap class loader: Bootstrap class loader is responsible to load all pre-defined java classes available in rt.jar file

=====

Note : rt.jar means Runtime Jar(java.lang, java.io, java.util etc all these packages related classes available in rt.jar file only

2.Extension class loader:Extension class loader is responsible to load java classes which are part of ext folder.

=====

C:\Program Files\Java\jre1.8.0_202\lib\ext

3.Application class loader:Application class loader is responsible to load our user defined classes.

=====

=====

Scanner:

=====

=>: Scanner class is available in java.util package.

=>: By using Scanner class we can read data from keyboard.

=====

StringTokenizer:

=====

=>: String Tokenizer is a pre-defined class available in java.util package.

=>: By using this class we can split String into multiple token based on delimiter.

=====

Java 1.8 Features

=====

=>: Java 1.8v introduced lot of new features in java.

=>: Java 1.8v new features changed java programming style.

Main Objectives of java 1.8v:

=====

=>: Simplify Java Programming.

=>: Enable Functional programming.

=>: Write more readable and concise code.

Java 1.8v Features:

=====

1.Interface Change

1.1) Default Methods

1.2) Static Methods

2. Functional Interface (@FunctionalInterface)

- 2.1) Predicate
- 2.2) Consumer
- 2.3) Supplier
- 2.4) Function

3. Lambda Expressions

4. Method References & Constructor References

5. Stream API

6. Optional Class

7. Spliterator

8. StringJoiner

9. Collections Framework changes

10. Date & Time API

Interface:

=====

=>: Interface means collection of abstract methods. (The method which doesn't contain body is called as Abstract method)

=>: A Class can implement interface using "implements".

=>: when a class is implementing interface its mandatory that class should implement all abstract methods of that interface otherwise class can't be compile.

=>: Here i am taking one interface with one abstract method. All the classes which are implementing that interface should override interface methods.

Ex:

=====

```
interface Vehicle {  
    public abstract void startVehicle();  
}  
  
class Car implements Vehicle {  
    public void startVehicle(){  
        // logic to start car  
    }  
}
```

```

    }

    class Bus implements Vehicle {

        public void startVehicle(){

            // logic to start car

        }

    }

    class Bike implements Vehicle {

        public void startVehicle(){

            // logic to start car

        }

    }

```

=>: If we add new method in interface then Car, Bike, Bus will fail at compile time.

=>: To overcome above problem we will use Default & Static methods.

Interface Changes:

=====

=>: Interface can have concrete methods from 1.8v

=>: Interface concrete method should be Default or Static.

=>: Interface default methods we can override in impl classes.

=>: Interface static methods we can't override in impl classes.

=>: We can write multiple default & static methods in interface.

=>: Default & Static method introduced to provide backward compatability.

EX: forEach() method added in java.util.Iterable interface as default method in 1.8v

Lambda Expression:

=====

=>: It is introduced in java 1.8v to enable Functional Programming.

=>: Java is called as Object Oriented Programming language. Everything will be represented using Classes and Objects.

=>: From 1.8v onwards java is also called as Functional Programming language.

=>: In OOP language classes & objects are main entities. We need to write methods inside the class only.

=>: Functional Programming means everything will be represented in the form of functions. Functions can be existed outside of the class. Functions can be stored into a reference variable. A function can be passed as a parameter to other methods.

=>: Lambda Expressions introduced in java to enable Functional Programming.

What is Lambda:

=====

=>: Lambda is an anonymous function.

- > no name
- > no access modifier
- > no return type

EX:1

=====

Normal method:

```
public void m1() {  
  
    s.o.p("Hi");  
  
}
```

Lambda Expression:

```
()->{s.o.p("Hi");}
```

Note: When we have a single line in body then curly braces are optional.

```
()-> s.o.p("Hi");
```

Ex:2

=====

Normal method:

```
public void add(int a, int b) {  
    s.o.p(a+b);  
}
```

Lambda Expression:

```
(int a, int b) -> { s.o.p(a+b) };  
                (or)  
(int a, int b) -> s.o.p(a+b);
```

(or)
(a,b)->s.o.p(a+b);

Ex:3

=====

Normal method:

```
public int getLength(String name) {  
    return name.length();  
}
```

Lambda Expression:

```
(String name) -> { return name.length() };  
    (or)  
(String name) -> return name.length();  
    (or)  
(name) -> return name.length();  
    (or)  
name -> name.length();
```

Functional Interface:

=====

=>: The interface which contains only one abstract method is called as Functional Interface.

=>: Functional Interfaces are used to invoke lambda expressions.

```
Runnable -----> run() method  
Callable -----> call() method  
Comparable -----> compareTo() method
```

=>: To represent one interface as functional interface we will use @FunctionalInterface annotation.

Ex:

=====

```
public interface MyInterface {  
    public void m1();  
}
```

Note: When we use @FunctionalInterface then our compiler will check interface contains only one abstract method or not.

=>: In java 8 several pre-defined functional interfaces are introduced.

=>: They are :

- 1.Predicate
- 2.Supplier
- 3.consumer

4.Function

=>: The above interfaces are available in java.util.function package.

1.Predicate :

=====

=>: It is pre-defined functional interface available in java.util.function package.

=>: It is used to check condition and return true or false.

=>: It will take only one parameter as input and return boolean as output.

=>: Predicate interface having only one abstract method that is test(T t) method.

```
=>:    interface Predicate {  
        boolean test(T t);  
    }
```

Predicate Joining:

=====

=>: To combine multiple predicates we will use Predicate joining.

=>: We have 2 methods in predicate joining.

=>: They are:

1.and() method

2.or() method

2.Supplier Functional Interface:

=====

=>: Supplier is a pre-defined functional interface available in Java.util.function package.

=>: It contains only one abstract method that is get() method.

=>: Supplier interface will not take any input, it will return the value.

3.Consumer functional interface:

=====

=>: Consumer is a pre-defined functional interface.

=>: It contains only one abstract method that is accept(T t) method.

=>: Consumer will take input but it won't return anything.

Note: In java-8 forEach() method is introduced. forEach(Consumer consumer) method will take consumer as parameter.

Q) What is the difference between supplier interface and consumer interface?

=>: Supplier will not take any input but it will returns the value whereas consumer will take input but it wont return anything.

4.Function functional interface:

=====

=>: Function is a pre-defined functional interface.

=>: Function interface having one abstract method that is apply(T r) method.

```
interface Function<R,T> {  
    R apply(T t);  
}
```

=>: It takes input and returns output.

Function Chaining:

=====

=>: It has 2 methods

```
1.andThen()  
2.compose()
```

Ex:

```
Function<Integer, Integer> f1 = n-> n*2;  
Function<Integer, Integer> f2 = n-> n*n*n;
```

```
System.out.println(f1.andThen(f2).apply(2)); // 4    64
```

Note(andThen): First f1 will execute and then result will be input for f2

```
System.out.println(f1.compose(f2).apply(2)); // 8    16
```

Note(compose): First f2 will execute and then result will be input for f1

StringJoiner:

=====

=>: java.util.StringJoiner class introduced in java 1.8v

=>: It is used to join more than one string with specified delimiter.

=>: We can concat prefix and suffix while joining strings using StringJoiner.

Note: StringJoiner sj = new StringJoiner(CharSequence delim);
 StringJoiner sj = new StringJoiner(CharSequence delim, CharSequence prefix,

```
CharSequence suffix);
```

Optional Class:

=====

=>: java.util.Optional classs is introduced in java 1.8v

=>: Optional class is used to avoid NullPointerExceptions in the program.

Q) What is NullPointerException ?

=>: When we perform some operation on null value then we will get
NullPointerException.

```
EX:      String s = null;
         s.length();    // NPE
```

=>: To avoid NullPointerExceptions we have to implement null check before
performing operation on the object like below.

```
String s = null;
if(s!=null) {
    System.out.println(s.length());
}
```

there 3 methods in optional class:-

```
empty()
of()
ofNullable()
```

when to use of() and when to use nullable():- If you know our object is return some
null values then use [of()] method

If you don't know, then use

[nullable()] method

Date & Time API Changes:

=====

=>: In java we have below 2 classes to represent Date.

1. java.util.Date
2. java.sql.Date

Note: When we are performing database operations then we will use java.sql.Date
class.

=>: For normal date related operations we will use java.util.Date class.

```
Date d = new Date();
System.out.println(d);
```

Note: When we create object for Date class then it will represent both Date & Time.

=>: If we want to get only date or only time then we need to format it using SimpleDateFormat class.

SimpleDateFormat:
=====

=>: SimpleDateFormat is a pre-defined class available in java.text package

=>: This class provided methods to perform Date conversions.
oo0o0o0

 Date to String conversion =====> String format(Date d)
 String to Date conversion =====> Date parse(String str)

Note: To overcome the problems of java.util.Date class java 1.8 introduced Date API changes.

=>: In java 1.8v new classes got introduced to deal with Date & Time functionalities.

- 1.java.time.LocalDate (It will deal with only Date)
- 2.java.time.LocalTime (It will deal with only Time)
- 3.java.time.LocalDateTime(It will deal with both date and time)

Stream API:
=====

=>: Stream API introduced in java 1.8v

=>: Stream API is used to process the data.

Note: Collections are used to store the data.

=>: Stream API is one of the major feature added in java 1.8v

=>: Stream in java can be defined as sequence of elements that comes from a source.

=>: Source of data for the Stream can be array or collection.

Few Important Points about Streams:
=====

=>: Stream is not a data structure.Stream means bunch of operations applied on source data.Source can be collection or array.

=>: Stream will not change original data structure of the source(It will just process the data given by the source).

Stream Creation:

=====

=>:In java we can create Stream in 2ways.

- 1.Stream.of(e1,e2,e3.....)
- 2.stream() method

Stream Operations:

=====

=>: Stream API provided several methods to perform operations on the data.

=>: We can divide Stream API methods into 2 types.

1.Intermediate Operational methods:Intermediate operational methods will perform operations on the stream and returns a new stream.

=====

=>: It is also called as Non-Terminal Operations or Processing Operations.

Ex: filter(), map(), flatMap(), distinct(),
limit(), sorted(), peek(), skip()

2.Terminal Operational Methods:Terminal Operational Methods will take input and provide result as output.

=====

Ex: count(), collect(), forEach(), min(),
max(), reduce(), toArray() findFirst(), findAny()

Note: After applying non-terminal operations we can apply terminal operations.

EX: name.stream().flatMap(x->x.stream()).forEach(System.out::println);

Filtering With Streams :

=====

=>: Filtering means getting required data from the original data.

EX: get only even numbers from the given numbers.

Ex: get employees whose salary is >= 25000.

=>: To Apply filter on the data, Stream API provided filter() method.

Ex: Stream filter(Predicate p)

Mapping Operations:

=====

=>: Mapping Operations are belongs to intermediate operations in the stream Api.

=>: mapping Operations are used to transform the stream elements and return transformed elements as new stream.

Q) What is flatMap(Function f) method ?

=>: It is used to flatten list of streams into single stream.

Slicing Operations with Stream :

=====

1.distinct() -----> To get unique elements from the stream

2.limit(long maxSize)----> To get elements from the stream based on given size.

3.skip(long n)-----> It is used to skip given number of elements from starting position of the stream.

Note: All the above 3 methods are comes under intermediate Operational Methods. They will perform operation and return new stream.

Matching Operations with Stream:

=====

1.anyMatch(Predicate p)-----> Atleast one should be matched

2.allMatch(Predicate p)-----> All should be matched

3.noneMatch(Predicate p)-----> No one should be matched

Note: All the above 3 methods are comes under terminal Operational Methods. They will perform operation and return result.

=>: The above methods are used to check the given condition and returns true or false value based on condition.

Collectors with Stream:

=====

=>: Collectors are used to collect data from stream.

Q) What is the difference between map() and flatMap() method ?

=>: map() function produces one output for one input value, whereas flatMap() function produces an arbitrary no of values as output (ie zero or more than zero) for each input value.

////////////////////////////////////
////////////////////////////////////

Total we have 5 SOLID principles

=====

1. S-> Single Responsibility principle :- every Java class must perform a single functionality

----- (i.e one class must have only one method).

2. O-> Open-Closed Principle:- the module should be open for "extension" but closed for "modification".

----- (if u want to do any modifications then we can only override it)

3. L-> Liskov Substitution:- Child class must be substitutable(replacable) for their Parent classes

----- (parent object must be replacable with child object)

4. I-> Interface Segregation principle:- (same as Single responsibility principles, but it is for interfaces)

----- * larger interface split into smaller ones.

* We should not force the client to use the methods that they do not want to override.

5. D-> Dependency Inversion Principle:- we must use abstraction (abstract classes and interfaces) instead of concrete implementations.

=====

Monolithic-Architecture:

=====

Monolithic applications are single-tiered, which means multiple components are combined into one large application. Consequently, they tend to have large codebases, which can be cumbersome(clumsy or gajibiji) to manage over time.

-> if one program component must be updated, other elements may also require rewriting, and the whole application has to be recompiled and tested.

Disadvantages:-

1. Large and complex applications may tend to have "large code bases"
2. Slow development
3. difficult to understand and modify
4. difficult to adapt new framework

Microservices:-

=====

Microservices allow a large application to be separated into smaller independent parts, with each part having its own realm of responsibility.

-> simply, splitting one large application into many services.

Advantages:-

1. Improve fault isolation: if one service encounters a fault or failure, we no need to change and deploy the entire system.
2. It will improve the team productivity
3. Deployment time is less compare to Monolithic.
4. Loosely coupled

Disadvantages:-

1. Developers may have to write extra code to ensure smooth communication between modules.

2. Coordinating deployments and managing version control across multiple services can be complex
3. Testing microservices involves complex scenarios, mainly when conducting integration testing across various services.

flow:-

=====

client -> API Gateway -> Microservices(here we have all Backend code)

Design pattern:-

=====

-> It is a description or template for how to solve a problem that can be used in many different situations.

-> It is a general repeatable solution to a commonly occurring problem in software design.

different design patterns in Microservices:

1. Service Registry
2. Circuit breaker
3. API Gateway
4. Event-Driven Architecture
5. Database per Service
6. Command Query Responsibility Segregation (CQRS)
7. Externalized Configuration
8. sega pattern
9. Bulkhead Pattern
10. Backends for Frontends (BFF)

=====

Java Contents:

JDK, JRE, JVM
Java features
data types and variables
operators
type casting
keywords
conditional statements
loops
Constructors
Super and This keywords

Oops
Interface and marker interface
Wrapper classes
Multi threading
Exception handling
arrays (1d and multi)
Recursions
Blocks (static, non static)
File handling
Strings (buffer, and builders)
serialization and de-serialization

Reflection API

Grbage collector
generics
var-args
enums(constants)
Inner class

Collection

Java 8

Advance java:-
JDBC
Servlet
JSP
Hibernate+JPA

Modules of Spring framework:-
Spring Core
Spring MVC
Spring AOP
Spring Boot
Spring data JPA
Spring Security
Spring Batch
REST

Microservices :-
1. REST TEMPLATE 2. Feign Client)
Eureka server

=====

All spring boot annotations:

=====

@SpringBootApplication == 3 annotations

@ComponentScan :- for scanning the packages

@EnableAutoConfiguration :- enables Spring Boot to auto-configure the application context. Therefore, it automatically creates and registers beans based on both the included jar files.

@Configuration :- a class-level annotation indicating that an object is a source of bean definitions(@configuration is the parent class for @bean(method level)).

Note: In spring boot (@SpringBootApplication) will come by default

* In spring only we will use these 3 annotation, But in springboot we will not use these 3 (instead will use @SpringBootApplication).

Stereotype annotation :

@Component (base class) == 3annotation

2. @Service -> business logic

3. @RestController(Json format response body) (controller + response body)
/ @Controller(returns view object(html, xml etc) format response body)

4. @Repository -> DB related

Spring Core related Annotations:

@Configuration :- to provide java based configuration

@Bean :- will use only inside the class which is annotated as [@configuration],
-> method level annotation

@Autowired :- for providing Dependency Injection [simply for object creation]

@Qualifier :- (eg: if we provide 2 @service provided in a project, then ioc container will get confusion to which one need to consider, to avoid this will use @Qualifier.

-> if ur sure to pic that particular bean (when u have multiple beans of same). -> Method level annotation

@Primary :- this is used to give higher preference to a bean when we have multiple beans of same type. (we can use this instead of @Qualifier) -> Class level annotation

-> It will set that particular bean as default bean

-> if ur unsure which one to use then u can use this.

@Lazy :- Create bean when we use it (or) when it only required still then(it will not execute) hold on..

@Value :- To get the values from Applications.properties file.
 (key = value) -> (server.port=8080) -> key=server.port and value=8080
 Eg:- @Value("\${server.port}") -> will give output as 8080

@PropertySource :- when we create our own custom application.properties file (eg: custom.properties) and if u want get values(use @value) from that file then will use this.

@ConfigurationProperties :- allows the developer to map the entire content of the applications.properties file to a POJO class using prefix of that. [eg: @ConfigurationProperties(prefix="server")]

@Profile :- Used to activate or deactivate specific bean configurations based on the active Spring profile.
 Provides a mechanism for managing different configurations for different environments (e.g., development, testing, production).
 . For example, you might have one set of configurations for your dev, Prod etc.
 (for dev one app.properties file and prod one more..)

@Scope :- which tells the application context about how that particular bean should be created and presented throughout the application lifecycle.
 eg:- @scope("singleton") -> only one bean is created and shared across entire application
 eg2:- @scope("prototype") -> A new instance of a bean is created when it is requested.

REST API related Annotations:

@RestController(@controller + response body) : it can provide response body as json formate

@RequestMapping(class or method level) : for providing base url of endpoint which we going to hit from postman

@GetMapping(fetching): for sending request without request boody(uses HTTP GET method)

@PostMapping(saving into DB): for sending request with request boody(uses HTTP POST)

@PutMapping(update or modify): for sending request with request boody with updated data uses HTTP PUT method), we should send the entire request body.

@DeleteMapping(deleting): for deleting specific record from DB

@PatchMapping(partial updation) : Fields that need to be updated by the user only updated without modifying the other field. (we should send the request body that for only required fields)

@RequestBody : to send the request body in the incoming request. (json)

@ResponseBody : we no need to use it, spring boot will provides bydefault
 -> Use [ResponseEntity<String>] as return Type, left side to method nam

@Valid : for making the conditions in entity class (like, @NotNull, @NotEmpty, @Min, @Max) we should use @valid in a method inside a controller)

@PathVariable: to specify one particular field(variable) in uri path(we should use inside {id}).

Eg : -> @GetMapping("employee/{id}/{name}")
 -> url= http://localhost:8080/employee/1/

@RequestParam: to extract data from the query parameters in the request URL.
 Query parameters are the key-value pairs that appear after the ? in a URL.

Eg : @GetMapping("/employee/") ; "
url= http://localhost:8080/employee?id=1&name=bhavani

Exception Handling Annotations:

@ControllerAdvice (or) @RestControllerAdvice
@ExceptionHandler

Spring Data JPA related annotations:

@Entity : It gives tells to the JPA provider that, this class should be treated as a table in the database.

@Table: if u want to specify the name of the table as u required in a DB.

@Column: If u want to specify the name of the column of the table as u required in a DB.

@Transactional(class or method level) : If the transaction becomes successful then the changes made to the database are committed, if any transaction fails(any exceptions are there), all the changes made to that particular transaction can be rollback and it will ensure that the database remains in a consistent state.
-> By default it will be provided by jpaRepository.

Entity class relationships

@OneToOne
@OneToMany
@ManyToOne
@ManyToMany

Security related Annotations:

@CrossOrigin

@Secured : uses role based security.

@PreAuthorize

@PermitAll

=====
=====

